> **Fuente original:** https://algoritmos-rw.github.io/tda_bg/material/guias/bt/

Backtracking

Contenidos

1. Ejercicio resuelto
2. Ejercicios propuestos

Ejercicio resuelto

(★★★★) El problema conocido como *Feedback Vertex Set (FVS)* indica: Dado un grafo (por simplificación, no dirigido), indicar vértices a eliminar de dicho grafo de tal manera que el grafo quede acíclico.

Implementar un algoritmo que, por **backtracking**, encuentre el mínimo FVS (es decir, el conjunto de menor cantidad de vértices a remover). Si bien es obvio, el grafo **no** cuenta con una primitiva `tiene_ciclos` (e, incluso, si la tuviera, sería poco conveniente usarla, recomendamos pensar por qué).

Solución

El problema en sí a priori no tendría una enorme diferencia con otros ejercicios de Bactracking, salvo por la necesidad de detectar ciclos. El tema aquí es que podemos aplicar una gran poda, que además es extremadamente evidente: si una componente conexa no presenta ciclos, no tiene ningún sentido analizar qué pasaría si sacamos a un vértice de dicha componente. Simplemente es estamos perdiendo el tiempo. Este tipo de podas medianamente evidentes que muestran *entender el problema* y no simplemente *saberse una formula de Backtracking* es algo que se espera que se realice. Restaría realmente mucho no hacerlo.

Entonces, a partir de esto se puede plantear de diferentes formas. Lo que vamos a plantear aquí es ir vértice por vértice del grafo, dentro de un conjunto de vértices que (hasta ahora) tenga sentido ir analizando. Entonces, cada vez que sacamos un vértice, calculamos nuevamente cuáles tienen sentido, y los que no (componente conexa sin ciclos) directamente los sacamos.

Se podría hacer también que cada vez que vamos a analizar un vértice, se chequee si la componente a la que perteence es acíclica, y en ese caso saltearlo. Esto sería correcto, se tomaría como correcto totalmente (a fin de cuentas, muestra entender qué mejora se puede hacer en función de entender el problema), pero esto puede mejorarse para evitar hacer una operación lineal por cada vértice en todo momento, e ir ya reduciendo.

Entonces, primero vamos a hacer una breve función que nos determine las componentes conexas de un grafo:

```
def componentes_conexas(grafo):
   visitados = set()
   componentes = []
   for v in grafo:
        if v not in visitados:
            componente = []
            dfs_comp(grafo, v, visitados, componente)
            componentes.append(componente)
    return componentes

def dfs_comps(grafo, v, visitados, componente):
    componente.append(v)
    visitados.add(v)
    for w in grafo.adyacentes(v):
        if w not in visitados:
            dfs_comps(grafo, w, visitados, componente)
```

Esto es una función lineal sobre el grafo (en vértices y aristas, $\mathcal{O}(V+E)$).

Ahora hacemos una breve función que nos determine si una componente tiene ciclos. La hacemos con BFS, únicamente porque la anterior la hicimos con DFS (por supuesto, hacerla con DFS estaría bien):

```python
def tiene_ciclo(grafo, componente):
    cola = Cola()
    origen = componente[0]
    cola.encolar(origen)
    visitados = set()
    visitados.add(origen)
    padres = {origen: None}
    while not cola.esta_vacia():
        v = cola.desencolar()
        for w in grafo.adyacentes(v):
            if w in visitados:
                if w is not padre[v]: # si es el padre, no es un ciclo
                    return True
            else:
                padres[w] = v
                visitados.add(w)
                cola.encolar(v)
    return False
```

Entonces nuestra idea es: iniciamos obteniendo las componentes conexas del grafo y ya filtrar las que no tienen ciclos. Luego, por cada vértice que aún está en una componente con ciclo, vemos qué pasa si lo sacamos (y recalculamos las componentes –> pueden cambiar porque podríamos estar sacando un vértice que en este punto sea un punto de articulación de lo que resta del grafo). Este último paso sería incluso mejorable si mantuviéramos cuenta de las componentes conexas del grafo, y cuando sacamos un vértice vemos las componentes que hay a partir de este (sin contarlo… digamos, similar a lo hecho en la función `componentes_conexas`). Esto implicaría un tanto más de programación que incluso no creemos que traiga muchas mejoras (y no esperamos que lo hagan porque justamente, implica mucho más de desarrollo para mantener todo en cuenta). Obviamente, en un TP podría ser conveniente explorar esta posibilidad, y analizar cuánto mejora una determinada poda.

```python
def fvs_rec(grafo_actual, vertices_posibles, sol_parcial, sol_minima):
    if len(vertices_posibles) == 0:
        if len(sol_parcial) < len(sol_minima):
            return sol_parcial[:]
        else:
            return sol_minima

    # La poda obvia de casi todo problema, si ya no somos mejores al óptimo actual, cortamos.
    if len(sol_parcial) >= len(sol_minima):
        return sol_minima

    # lo saco para no considerarlo
    # es más fácil que ir por índices, siendo que vamos a ir recalculando
    posible = vertices_posibles.pop()

    solucion_minima = fvs_rec(grafo_actual, vertices_posibles, sol_parcial, sol_minima)

    adyacentes = grafo_actual.adyacentes(posible)
    grafo_actual.borrar_vertice(posible)
    nuevos_posibles = potenciales(grafo_actual)
    sol_parcial.append(posible)
    solucion_minima = fvs_rec(grafo_actual, nuevos_posibles, sol_parcial, solucion_minima)

    # Backtrack para futuros llamados. No hice una copia porque eso es más costoso que simplemente
    # sacar un vértice y volverlo a poner
    vertices_posibles.append(posible)
    grafo_actual.agregar_vertice(posible)
    for v in adyacentes:
        grafo_actual.agregar_arista(posible, v)

    return solucion_minima


def potenciales(grafo):
    nuevos_posibles = []
    for comp in componentes_conexas(grafo):
        if tiene_ciclo(grafo, comp):
            for v in comp:
                nuevos_posibles.append(v)
    return nuevos_posibles

def fvs(grafo):
    # se podria hacer una copia del grafo para no estarlo modificando
    # la solución mínima inicial la consideramos una con todos los vértices, que es una solución trivial
    # (y seguro mejorable)
    return fvs_rec(grafo, potenciales(grafo), [], grafo.vertices())
```

Ejercicios propuestos

1. (★★) Implementar por backtracking un algoritmo que, dado un grafo no dirigido y un numero n menor a $|V|$, devuelva si es posible obtener un subconjunto de n vertices tal que ningun par de vertices sea adyacente entre si.

2. (★★) Implementar un algoritmo que reciba un grafo y un número n que, utilizando backtracking, indique si es posible pintar cada vértice con n colores de tal forma que no hayan dos vértices adyacentes con el mismo color.

3. (★) Dado un tablero de ajedrez $n \times n$, implementar un algoritmo por backtracking que ubique (si es posible) a n reinas de tal manera que ninguna pueda comerse con ninguna.

4. (★★) Implementar un algoritmo que dado un Grafo no dirigido nos devuelva un conjunto de vértices que representen un máximo Independent Set del mismo.

5. (★★) Un camino hamiltoniano, es un camino de un grafo, que visita todos los vértices del grafo una sola vez. Implementar un algoritmo por backtracking que encuentre un camino hamiltoniano de un grafo dado.

6. (★) Dada una matriz de 9x9, implementar un algoritmo por backtracking que llene la matriz con números del 1 al 9, dadas las condiciones del Sudoku (si es posible). Las condiciones son: (i) Las celdas están dispuestas en 9 subgrupos de 3x3. (ii) Cada columna y cada fila no puede repetir número. (iii) Cada subgrupo de 3x3 no puede repetir número.

7. (★) Implementar un algoritmo de backtracking que, dado una pieza de caballo en un tablero de ajedrez, determine los movimientos a realizar para que el caballo logre pasar por todos los casilleros del tablero una única vez. Recordar que el caballo mueve en forma de L (dos casilleros en una dirección, y un casillero en forma perpendicular).

8. (★★★) Implementar un algoritmo de backtracking que, dados dos grafos, determine si existe un Isomorfismo entre ambos.

9. (★) Se tiene una lista de materias que deben ser cursadas en el mismo cuatrimestre, cada materia está representada con una lista de cursos/horarios posibles a cursar (solo debe elegirse un horario por cada curso). Cada materia puede tener varios cursos. Implementar un algoritmo de backtracking que devuelva un listado con todas las combinaciones posibles que permitan asistir a un curso de cada materia sin que se solapen los horarios. Considerar que existe una función son_compatibles(curso_1, curso_2) que dados dos cursos devuelve un valor booleano que indica si se pueden cursar al mismo tiempo.

10. (★) Implementar un algoritmo tipo Backtracking que reciba una cantidad de dados n y una suma s. La función debe devolver todas las tiradas posibles de n dados cuya suma es s. Por ejemplo, con $n = 2$ y $s = 7$, debe devolver `[[1, 6], [2, 5], [3, 4], [4, 3], [5, 2], [6, 1]]`. ¿De qué complejidad es el algoritmo en tiempo? ¿Y en espacio?

11. (★) Escribir un algoritmo que, utilizando backtracking, dada una lista de enteros positivos L y un entero n devuelva todos los subconjuntos de L que suman exactamente n.

12. (★) Modificar el algoritmo anterior para que, dada una lista de enteros positivos L y un entero n, devuelva un subconjunto de L que sume exactamente n, o, en caso de no existir, que devuelva el subconjunto de suma máxima sin superar el valor de n.

13. (★★) Un Vertex Cover de un Grafo G es un conjunto de vértices del grafo en el cual todas las aristas del grafo tienen al menos uno de sus extremos en dicho conjunto. Por ejemplo, el conjunto de todos los vértices del grafo siempre será un Vertex Cover. Implementar un algoritmo que dado un Grafo no dirigido nos devuelva un conjunto de vértices que representen un mínimo Vertex Cover del mismo (es decir, que sea el conjunto de tamaño mínimo).

14. (★★★) Un set dominante (Dominating Set) de un grafo G es un subconjunto **D** de vértices de G, tal que para todo vértice de G: o bien (i) *pertenece a D*; o bien (ii) es *adyacente a un vértice en D*. Implementar un algoritmo que reciba un Grafo, y devuelva un dominating set de dicho grafo con la mínima cantidad de vértices.

15. (★★★) Un bodegón tiene una única mesa larga con W lugares. Hay una persona en la puerta que anota los grupos que quieren sentarse a comer, y la *cantidad de integrantes* que conforma a cada uno. Para simplificar su trabajo, se los anota en un vector P donde $P[i]$ contiene la cantidad de personas que integran el *grupo i*, siendo en total n grupos. Como se trata de un restaurante familiar, las personas sólo se sientan en la mesa si *todos los integrantes de su grupo* pueden sentarse. Implementar un algoritmo que, **por backtracking**, obtenga el conjunto de grupos que ocupan la mayor cantidad de espacios en la mesa (o en otras palabras, que dejan la menor cantidad de espacios vacíos).

16. (★★★) Para ayudar a personas con problemas visuales (por ejemplo, daltonismo) el gobierno de Agrabah decidió que en una misma parada de colectivo nunca pararán dos colectivos que usen el mismo color. El problema es que ya saben que eso está sucediendo hoy en día, así que van a repintar todas las líneas de colectivos. Por problemas presupuestarios, desean pintar los colectivos con la menor cantidad posible k colores diferentes. Como no quieren parecer un grupo de improvisados que malgasta los fondos públicos, quieren hacer un análisis para saber cuál es ese mínimo valor para cumplir con lo pedido (pintar cada línea con alguno de los k colores, de tal forma que no hayan dos de mismo color coincidiendo en la misma parada). Considerando que se tiene la información de todas las paradas de colectivo y qué líneas paran allí, modelar el problema utilizando grafos e implementar un algoritmo que determine el mínimo valor k para resolver el problema. Indicar la complejidad del algoritmo implementado.

17. (★★★) Se tiene una matriz donde en cada celda hay submarinos, o no, y se quiere poner faros para iluminarlos a todos. Implementar un algoritmo que dé la cantidad mínima de faros que se necesitan para que todos los submarinos queden iluminados, siendo que cada faro ilumina su celda y además todas las adyacentes (incluyendo las diagonales), y las directamente adyacentes a estas (es decir, un “radio de 2 celdas”).

18. (★★★) Implementar un algoritmo que, por backtracking, obtenga la cantidad total de posibles ordenamientos topológicos de un grafo dirigido y acíclico.

19. (★★) Implementar un algoritmo que, utilizando **backtracking**, resuelva el problema del cambio (obtener la forma de dar cambio en la mínima cantidad de monedas) con una nueva restricción: no se tiene una cantidad indefinida de cada moneda, sino una cantidad específica (y esto hace que pueda no haber solución). Suponer que la función a invocar es `cambio(n, monedas, cantidad_x_monedas)`, donde `n` sea el valor a devolver en cambio, `monedas` sea una lista ordenada de los valores de las monedas, y `cantidad_x_monedas` un diccionario.